ACTIVIDAD 01 / APLICACIONES MÓVILES I

Express GO
Selección del framework móvil

Pedidos de madrugada en Santa Cruz
Práctica Flutter con el prototipo ConectaCruz

Rodrigo Ledezma Sanchez
Universidad Privada Domingo Savio

Recorrido de la defensa

01 Contexto y alcance

Necesidades del comercio local y objetivos del prototipo.

02 Comparación y resultados

Tres alternativas, cinco criterios y dos escenarios de sensibilidad.

03 Evidencia y próximos pasos

Primera pantalla en Android, catálogo actual y plan de validación.

Problema y pregunta de investigación

¿Qué framework conviene a Express GO con un presupuesto limitado?

Comida de 00:00 a 06:00

El cliente necesita saber dónde pedir de madrugada. En el escenario del caso, menú, horarios y disponibilidad se reparten entre redes y WhatsApp.

Objetivo de la investigación

Elegir una tecnología para catálogo, pedidos y reservas de recojo o entrega. Demostrar en Android la primera pantalla exigida.

El problema que aborda Express GO

De 00:00 a 06:00 en Santa Cruz

Express GO reúne pocos negocios gastronómicos de Santa Cruz de la Sierra. De madrugada, el cliente necesita saber cuáles atienden, qué platos quedan y qué zonas cubren.

Información dispersa

En el escenario de trabajo, un negocio publica precios en redes y confirma disponibilidad por WhatsApp. Si el menú cambia o la consulta se pierde, el cliente puede intentar pedir un plato agotado.

Qué resolvería la aplicación

Mostrar locales abiertos, precio y disponibilidad en un catálogo común. Permitir una consulta y una reserva de pedido para recojo o entrega. El caso debe validarse con comercios piloto.

Ejemplo de pedido de madrugada

02:10 · Buscar

Una persona termina su turno en Santa Cruz y busca comida. Express GO muestra locales que todavía aceptan pedidos y descarta los que están cerrados.

02:12 · Consultar y reservar

La persona compara combos, verifica el precio y consulta la zona de entrega. Después elige un horario de recojo y envía la solicitud. El estado queda como pendiente.

02:15 · Confirmar

El negocio confirma el producto y el horario. Si agotó el combo, informa el cambio y el cliente decide si elige otro. Los horarios ilustran el flujo propuesto.

Alcance y objetivos del prototipo

Caso de negocio

Express GO plantea un catálogo de gastronomía nocturna, consultas y reservas de pedido.

Prototipo de la consigna

ConectaCruz muestra la bienvenida y abre un catálogo local con seis negocios de ejemplo.

Alcance del piloto futuro

Aplicar el recorrido al menú nocturno de Express GO y probar confirmaciones con comercios.

Tres alternativas para el mismo alcance

Flutter

Dart y widgets compartidos para Android e iOS. El proyecto de la asignatura ya utiliza este entorno.

React Native

Componentes para Android e iOS con JavaScript y React. Su incorporación depende de la experiencia del equipo.

Desarrollo nativo

Kotlin para Android y Swift con SwiftUI para iOS. Cada plataforma tiene su propio proyecto de interfaz.

Qué aportan las fuentes

Documentación oficial

Flutter y React Native documentan el trabajo entre plataformas. Android Developers y Apple describen Kotlin y SwiftUI para sus respectivos entornos. Estas fuentes respaldan capacidades técnicas.

Estudio comparativo

Ilovan (2026) comparó versiones de una misma aplicación y observó diferencias según la tarea y la plataforma. Es una prepublicación externa al proyecto.

Supuestos del caso

El horario nocturno, el presupuesto limitado, la experiencia previa en Dart y el uso de WhatsApp pertenecen al escenario de Express GO. Los puntajes son estimaciones para ese caso.

Ventajas y límites de Flutter

Ventaja para Express GO

Dart y los widgets permiten reutilizar las pantallas de catálogo, consulta y reserva entre Android e iOS. La bienvenida de ConectaCruz sirve como referencia para empezar la implementación.

Trabajo que sigue siendo necesario

Hay que configurar cada plataforma, probar en dispositivos reales y comprobar que los paquetes elegidos funcionen en ambos sistemas. Compilar iOS requiere un entorno macOS.

Cuándo reevaluar la elección

Si una función futura exige integración nativa muy especializada, conviene medir el esfuerzo de implementar y mantener esa integración antes de extender el alcance.

Ventajas y límites de React Native

Ventaja para un equipo con React

Comparte componentes y lógica entre Android e iOS. Un equipo que ya programa en React y JavaScript puede aprovechar ese conocimiento para construir la lista y el detalle.

Costo para este equipo

La curva de aprendizaje incluye JavaScript, React y su forma de manejar estado y navegación. En la evaluación se valoró el avance que ya existía en Dart.

Cuándo sería preferible

Si el equipo cuenta con experiencia en React y conserva los demás supuestos de la matriz, un aumento en la nota de aprendizaje puede cambiar el resultado.

Ventajas y límites del desarrollo nativo

Android e iOS

En el enfoque comparado, se desarrolla con Kotlin para Android y Swift con SwiftUI para iOS. Cada versión usa herramientas e interfaces propias de la plataforma.

Ventaja técnica

El acceso directo a las herramientas de cada sistema puede ayudar a integrar funciones específicas. La evidencia comparativa muestra ventajas en algunas operaciones, según la carga.

Costo para el prototipo

Dos implementaciones requieren repetir parte del diseño, las correcciones y las pruebas. Con un presupuesto limitado, ese esfuerzo pesa aunque ambas plataformas estén cubiertas.

Cómo se construyó la evaluación

Costo y rendimiento

30 % para esfuerzo de desarrollo y mantenimiento. 20 % para respuesta esperada del catálogo.

Comunidad y soporte

15 % para recursos de apoyo. 20 % para cobertura de Android e iOS.

Aprendizaje

15 % para el esfuerzo de incorporación del equipo. La escala va de 1 a 5.

Por qué se asignaron esas notas

Costo y rendimiento

Costo: Flutter 5, React Native 5 y nativo 2 por el esfuerzo de mantener dos interfaces. Rendimiento esperado: 4, 4 y 5. La diferencia se apoya en evidencia externa y debe medirse en el catálogo.

Comunidad y plataformas

Se asignó 4 a las tres opciones en comunidad porque la revisión encontró guías y recursos, sin una medida comparable para separarlas. Todas obtuvieron 5 en cobertura de Android e iOS.

Aprendizaje

Flutter 4 por el entorno y el código Dart disponibles. React Native 3 por la incorporación de React. Nativo 2 porque el equipo tendría que incorporar dos entornos de desarrollo.

Ejemplo de cálculo ponderado

Escala común

1 representa una condición desfavorable, 3 una situación intermedia y 5 una favorable. Cada criterio tiene un peso según su importancia para Express GO. Los pesos suman 100 %.

Cálculo paso a paso

Flutter tiene nota 5 en costo, de modo que 30 × 5 ÷ 5 = 30 puntos. En rendimiento tiene nota 4, por lo que 20 × 4 ÷ 5 = 16 puntos.

Total por alternativa

El total es la suma de los cinco aportes. Por ejemplo, Flutter suma 30 + 16 + 12 + 20 + 12 = 90. El resultado queda en una escala de 0 a 100.

Matriz ponderada

Criterio	Peso	Flutter	React Native	Nativo
Costo	30	30	30	12
Rendimiento	20	16	16	20
Comunidad	15	12	12	12
Soporte Android e iOS	20	20	20	20
Curva de aprendizaje	15	12	9	6
Total / 100	**100**	**90**	**87**	**70**

Aporte = peso × calificación ÷ 5. La escala de calificación va de 1 a 5.

La experiencia del equipo cambia la decisión

3 puntos

de ventaja para Flutter en el escenario base

Costo y plataformas

Flutter y React Native empatan en costo. Las tres opciones cubren Android e iOS.

Escenario con experiencia en React

React Native sube de 87 a 93 si su nota de aprendizaje pasa de 3 a 5.

La matriz favorece la experiencia disponible para este prototipo.

Dos escenarios de sensibilidad

Equipo con experiencia en React

Si React Native sube de 3 a 4 en aprendizaje, empata con Flutter. Con 5, llega a 93 puntos.

Mayor peso del rendimiento

Al trasladar 10 puntos de costo a rendimiento, Flutter queda en 88, React Native en 85 y nativo en 76.

Son simulaciones de la matriz. Los valores dependen de las prioridades y calificaciones adoptadas.

Qué puede cambiar con pruebas reales

El desarrollo nativo merece consideración

El estudio registró ventajas nativas en algunas operaciones. El orden varió según tarea y plataforma.

El emulador verifica la interfaz

La demostración confirma el arranque de la bienvenida en Android. Quedan por medir memoria, arranque y fluidez en teléfonos físicos.

Validación para ConectaCruz

Primero se evaluaría el catálogo con imágenes. Después se probarían los flujos de pedido y reserva.

Instalación del entorno en Windows

SDK de Flutter y PATH

Se descomprimió el SDK en C:\src\flutter y se añadió C:\src\flutter\bin al PATH del usuario. Una terminal nueva recoge el cambio y puede ejecutar flutter --version.

Herramientas para Android

Se usó Android Studio con su JDK. El SDK de Android proporciona las plataformas y herramientas de compilación. Las herramientas de línea de comandos permiten gestionar las licencias.

Emulador

Se configuró el dispositivo virtual Pixel 6 Pro. Flutter puede listar los dispositivos disponibles y elegir ese Android como destino para instalar y ejecutar la app.

Cómo se verificó flutter doctor

Licencias y diagnóstico

flutter doctor --android-licenses permite revisar las licencias del SDK. flutter doctor -v comprueba el entorno e indica qué componente falta o necesita atención.

Resultado registrado

La salida incluye Flutter 3.47.9 en el canal estable, Dart 3.13.7, Android SDK 37.0.0 y licencias de Android aceptadas. El diagnóstico termina con «No issues found!».

Evidencia y alcance

El archivo evidencias_actividad_01/flutter_doctor.txt conserva la salida detallada. Un entorno listo para compilar todavía requiere ejecutar el proyecto para confirmar que la interfaz aparece.

Cómo se ejecutó el prototipo

Preparar el proyecto

Desde la carpeta del repositorio, flutter pub get prepara las dependencias descritas en pubspec.yaml. La configuración de Android se encuentra en la carpeta android.

Elegir el destino

Con el dispositivo virtual iniciado, flutter devices permite identificar el emulador. Al seleccionar ese destino, Flutter compila la app y la inicia en Android.

Qué demuestra la captura

La pantalla debe mostrar el título ConectaCruz y el botón «Ver negocios». Una captura documenta el estado de esa pantalla y debe complementarse con una prueba de la navegación.

ConectaCruz ejecutado en Android

Inicio de la aplicación

Scaffold organiza el ícono, el título, la descripción y «Ver negocios».

Acción verificada

Al pulsar el botón, el emulador muestra el catálogo con los negocios de ejemplo.

Relación con Express GO

Este recorrido sirve de base para abrir el menú nocturno del negocio.

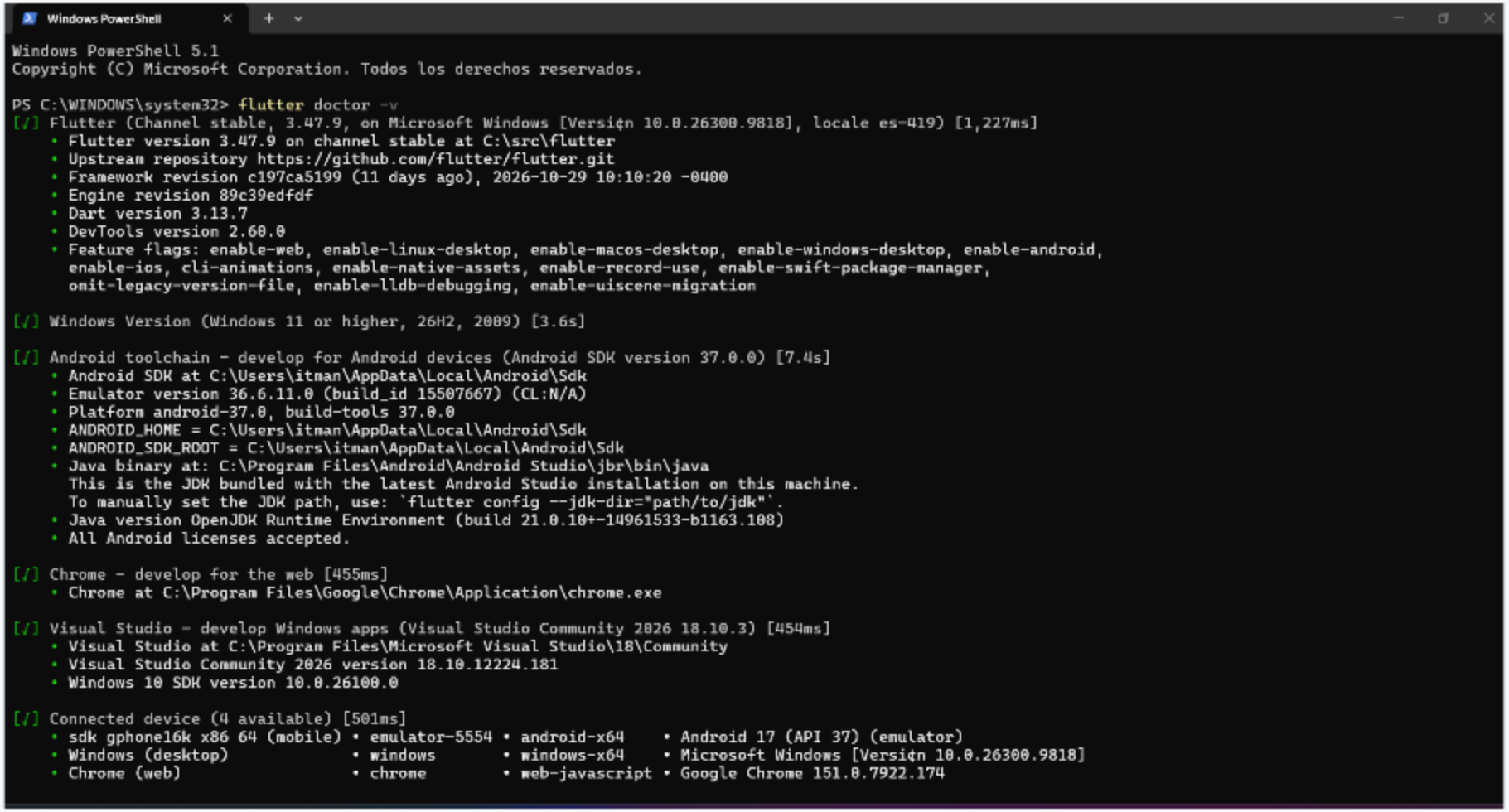
01 Bienvenida



02 Catálogo

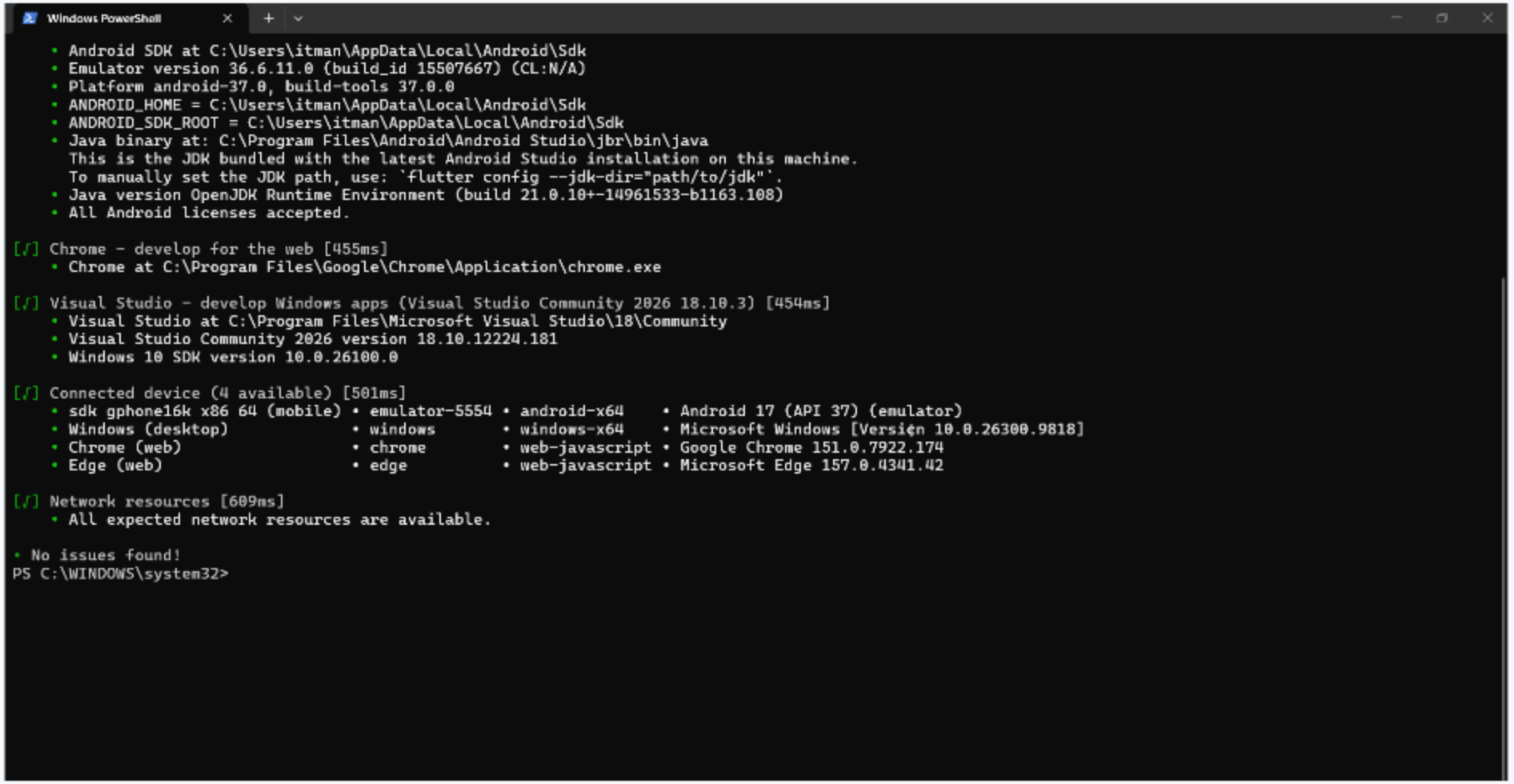


Evidencia del entorno Flutter y Android



La salida muestra Flutter 3.47.9, Dart 3.13.7, Android SDK 37.0.0 y licencias aceptadas.

Evidencia del emulador y el diagnóstico



La salida registra el emulador Android y termina con «No issues found!».

Cómo se compone la bienvenida

MaterialApp y Scaffold

MaterialApp configura el tema y elige BienvenidaPage como pantalla inicial. Scaffold da una estructura Material a esa pantalla y contiene el cuerpo de la interfaz.

SafeArea y Column

SafeArea evita zonas ocupadas por el sistema. Center y Padding centran y separan el contenido de los bordes. Column ordena de arriba abajo el ícono, el título, la descripción y el botón.

Elementos de interacción

Text comunica el nombre y el propósito. ElevatedButton.icon combina un texto visible con un ícono. El callback onPressed ejecuta la navegación al pulsar.

Qué hace «Ver negocios»

Entrada

El usuario inicia en BienvenidaPage. Al pulsar el botón, onPressed llama a Navigator.of(context).push con una MaterialPageRoute.

Destino

La ruta crea CatalogoPage y la coloca encima de la bienvenida. El catálogo construye tarjetas a partir de los seis negocios de demostración.

Regreso

La acción Atrás retira la ruta del catálogo y vuelve a la bienvenida. Esta estructura mantiene separadas la primera pantalla y la lista de negocios.

Cómo funciona el catálogo actual

Modelo Negocio

Cada registro contiene nombre, rubro, descripción e imagen. Hay seis ejemplos locales.

Pantalla CatalogoPage

ListView.builder construye las tarjetas del catálogo conforme se necesitan.

Tarjeta NegocioCard

Presenta los datos y muestra una alternativa visual si falla la imagen de red.

Del botón a la pantalla de negocios

Fragmento de lib/main.dart

```
onPressed: () {
  Navigator.of(context).push(
    MaterialPageRoute<void>(
      builder: (_) => const CatalogoPage(),
    ),
  );
},
```

La acción del botón

onPressed se ejecuta al pulsar «Ver negocios».

La ruta de Material

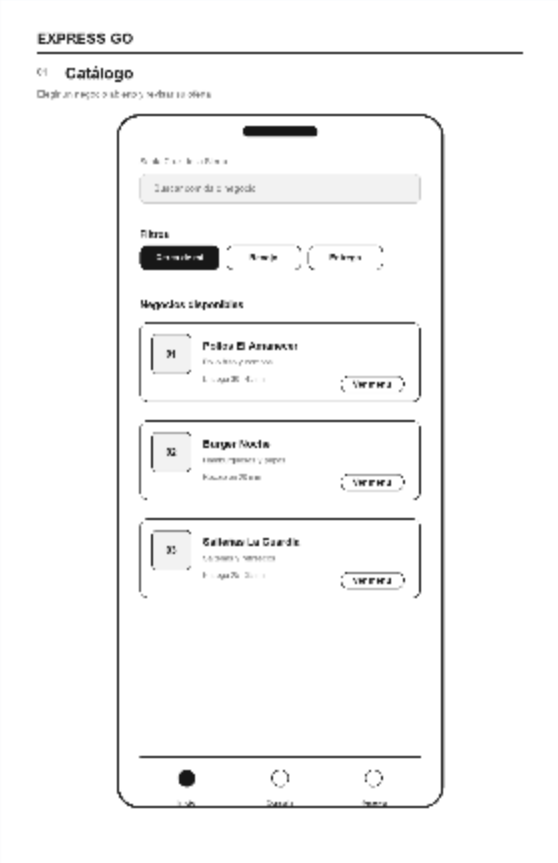
El builder crea CatalogoPage al abrir la ruta.

El historial de pantallas

Atrás vuelve a BienvenidaPage.

Propuesta de interfaz para Express GO

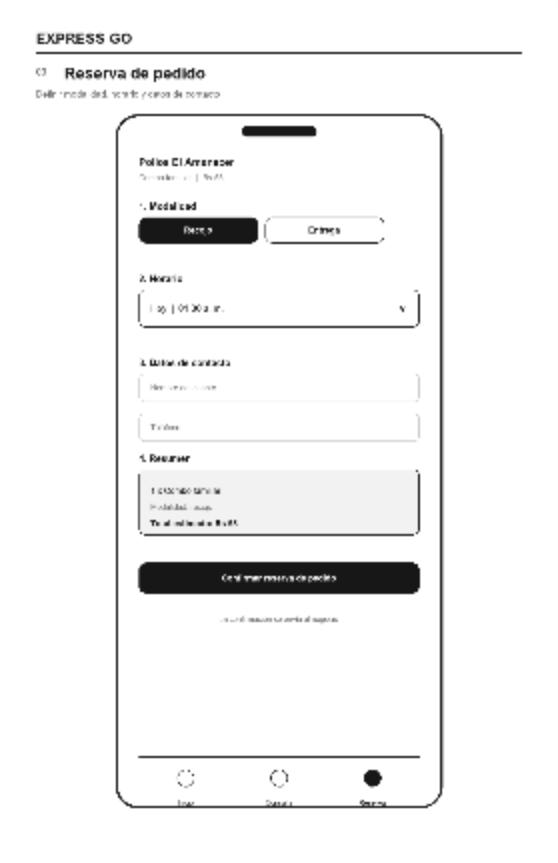
Catálogo nocturno



Consulta al local



Reserva de pedido



Bocetos de Express GO para validar el flujo con comercios piloto.

Catálogo, consulta y reserva en Express GO

Catálogo nocturno

Mostrar locales abiertos, precio, disponibilidad y modalidad de recojo o entrega. Un plato agotado debe verse como no disponible antes de iniciar el pedido.

Consulta y reserva de pedido

Resolver dudas sobre ingredientes, zona o tiempo. Registrar el producto, el local y el horario elegido. El negocio debe aceptar o rechazar la solicitud.

Confirmación y fallos de conexión

Mostrar los estados pendiente, confirmado y rechazado. Si se corta la red, informar si la solicitud llegó y evitar que el reintento genere un pedido duplicado.

Qué puede verificarse en la prueba

Pantalla de entrada

La prueba de widgets inicia ConectaCruzApp y comprueba que aparece el título. Antes de pulsar «Ver negocios», la primera tarjeta del catálogo no está visible.

Navegación y lista

La prueba pulsa el botón, comprueba que aparece Sabor Cruceño, valida que hay seis negocios de ejemplo y se desplaza para buscar la tarjeta Arte Guaraní.

Regreso a la bienvenida

La prueba usa Atrás y espera volver a ver el botón «Ver negocios». Este caso cubre la ruta inicial. Los pedidos, las reservas y el rendimiento en teléfonos reales tienen pruebas distintas.

Plan de validación

Navegación y conexión

Probar bienvenida, catálogo y detalle en equipos Android con conexión normal, lenta y sin conexión.

Arranque y desplazamiento

Medir varias ejecuciones con el mismo conjunto de negocios y registrar la mediana.

Pedidos y reservas

Comprobar confirmaciones y reintentos con comercios piloto antes de ampliar a iOS.

Decisión para el prototipo

Flutter

Flutter permite empezar el catálogo nocturno de Express GO con la base en Dart.

El próximo paso es validar horarios, disponibilidad y confirmación con negocios piloto.

Entrega y evidencias en GitHub

Investigación

El informe en PDF reúne introducción, métodos, resultados y discusión. Incluye índice, matriz ponderada, referencias en APA 7 y las limitaciones de la comparación.

Entorno y app

El repositorio conserva el diagnóstico de Flutter, la captura de la bienvenida en el emulador y el código de la ruta hacia el catálogo.

Defensa grabada

Para el video de hasta cinco minutos, seguir las notas con tiempo de las diapositivas principales. Las láminas de detalle amplían la lectura y sirven de apoyo a las preguntas.

Fuentes de la comparación

Android Developers
Android’s Kotlin-first approach

Apple Developer
Hello, SwiftUI

Flutter Docs
Build for and integrate with multiple platforms

Ilovan (2026)
Cross-platform vs native mobile development · arXiv

Meta Platforms (2026)
Introduction · React Native